

Mobile App — Scope & Access Strategy

A SaaS-owner-level decision doc: who gets to log into the mobile app, and what the Student experience should contain. Written against what the backend already supports today — face-recognition attendance, lecturer-side session check-in, QR bus boarding, a driver dashboard — so this isn't a wishlist, it's scoped to what's buildable now vs. later.

SECTION 1

Who Gets Mobile Login

ROLE	MOBILE LOGIN?	WHY
Student	Yes — primary user	The mobile app <i>is</i> the student's daily interface: attendance check-in, bus boarding, notifications. This is the app's main reason to exist.
Faculty	Yes — primary user	Attendance sessions are inherently mobile: a lecturer starts a session and walks the room. The backend already has <code>LecturerCheckInView</code> / <code>LecturerStartSessionView</code> built around this. Without Faculty on mobile, geofenced/QR attendance doesn't work at all.
Bus Driver / Conductor	Yes — specialized login	Not a new role — an individual Support Staff account assigned as <code>driver</code> on a route. Needs a stripped-down mobile view: today's route, stop list, expected boarding counts (<code>BusDriverDashboardView</code> already says " <i>Conductor/Driver dashboard view on mobile app</i> " in its own docstring). This person should not see anything else in the system.
Department Head	Not in v1	Leave-approval and announcement actions are low-frequency and fine on web. Reconsider for v2 as a lightweight "approvals & alerts" view — see Section 4.
Support Staff (non-driver)	Not in v1	Librarian/warden/store-keeper workflows (issuing books, allocating rooms) are desk-bound, form-heavy, better on web.
Management / Administrator	No	Payroll, fee reconciliation, audit logs, bulk directory management — none of this belongs on a phone. Full trust doesn't mean full mobile exposure; it means these two roles run the college from the web console.
SaaS Admin	No	Platform-level (tenant provisioning, billing, subscriptions). Entirely out of scope for a <i>tenant's</i> mobile app — this is your own internal tooling, not something a college's app ships with.

Rule of thumb: mobile login is for people who need the app *while moving around campus* (students between classes, faculty in the classroom, drivers on the route) — not for people who need a *keyboard and a spreadsheet* (admin/finance/HR work). If a workflow is a form with 8 fields, it's a web workflow.

SECTION 2

What Students See on Mobile

Design principle: mobile is for **glance, notify, and check-in** — not for data management. Every screen below answers "what do I need to know or do *right now*," not "let me review a report."

Home / Dashboard

- Today's classes at a glance (next class, room, time)
- Attendance % this month, with a visual nudge if it's dropping toward a shortage threshold
- Fee due banner if a payment is outstanding (tap to pay)
- Unread announcements count

Attendance (mobile-exclusive value)

- Face-recognition check-in with liveness challenge (already built: `FaceRegistrationView`, `LivenessChallengeView`, `MarkAttendanceView`) — the single feature that most justifies having a mobile app at all
- Attendance history (calendar/list view)
- "Request manual attendance" if biometric check-in failed (`StudentRequestManualAttendanceView` already exists for this)

Schedule

- Today/This-week timetable, not the full-semester grid — that's a web view

Exams

- Upcoming exam schedule (date, room, seat if assigned)
- Results, once published

Assignments

- List with due dates, push reminder as deadline approaches
- Submit via phone camera or file picker — genuinely easier on mobile than desktop for a photographed handwritten assignment

Fees

- Dues, due dates, payment history
- Pay now (existing Razorpay integration — `CreatePaymentOrderView`)

Bus Tracking

- Live map of their subscribed route
- **Scan-to-board QR** (`BusBoardingScanView`) — a mobile-only action, there's no web equivalent that makes sense
- Push notification: "bus is 5 minutes from your stop"

Hostel

- Room/allocation status
- Raise a maintenance complaint (photo attachment — mobile-native)

TPO / Placements

- Browse open drives, apply, track application status
- Push notification when a new drive matching their department opens

Library

- Search catalog, view currently issued books, due-date reminders
- Not: reservation queues or catalog *management* — that's staff/web work

Announcements

- Feed + push notifications — likely the single highest-engagement screen

Profile

- View-only profile, change password
- **Biometric consent management** — since face-recognition attendance processes biometric data, the mobile app needs an explicit consent/opt-out screen on first login before enrolling a face, tied into the DPDP consent log system already in the backend (`consent_given` fields, biometric notice flow). Don't silently start capturing face data.

Deliberately NOT on student mobile: anything about other students or staff · analytics, reports, directories · bulk actions of any kind · settings that affect anyone but themselves.

SECTION 3

What Faculty See on Mobile (brief — not the ask, but load-bearing for Section 1)

Scoped tightly around *running* a class session, not managing one:

- Start/end an attendance session for a lecture, see live check-in count (`LecturerStartSessionView` , `LecturerAttendanceStatusView`)
- Approve/reject manual attendance requests from students who failed biometric check-in (`LecturerManualRequestsView`)
- Today's schedule
- Apply for leave, see leave balance
- Push notifications for announcements

Everything else Faculty can do on web (grading, assignment creation, full analytics) stays web-only — those are desk tasks.

SECTION 4

Phasing

- **v1 (build now):** Student + Faculty + Bus Driver/Conductor mobile access, scoped as above.
- **v2 (reconsider once v1 is live):** a lightweight "Approvals & Alerts" mobile view for Department Head/Management — leave approvals, announcement broadcast, push alerts only. Not a shrunk version of the web dashboard; a purpose-built glance screen.
- **Never on tenant mobile:** Management/Administrator's data-management workflows (payroll, fee reconciliation, audit logs, bulk directory edits), and SaaS Admin entirely.

SECTION 5

One Open Question Worth Deciding Early

Should Faculty and Student share one app binary (role-detected after login, like the web app already does) or ship as two separate apps? The web app's existing pattern — one login, role resolved server-side, UI adapts — argues for **one app**, since it avoids duplicating auth/tenant-resolution logic and matches how the web role/module access design already works (see *CampusNexus_User_Manual.pdf*). The Driver/Conductor view would just be another role-detected UI state within the same app, not a third binary.